

Caso pedagógico 07 — Gateway reverso (Nginx) e exposição controlada em VPS

Data: 16/12/2025

Contexto: deploy de microsserviços em uma VPS (Oracle Cloud) com um **gateway Nginx** na borda e correção de bloqueios de firewall

1) Objetivo pedagógico

Este caso existe para que você consiga **replicar o padrão de produção** no seu próprio projeto:

- expor **apenas uma porta pública** (HTTP/HTTPS) na VPS;
 - usar um **gateway reverso** para rotear `/auth`, `/users`, `/events`, etc.;
 - diagnosticar corretamente quando “funciona por SSH, mas não funciona externamente”;
 - entender a diferença entre firewall **na VM** (iptables) e firewall **na nuvem** (Security List / VCN);
 - preparar o caminho para HTTPS com **Certbot** (quando for o momento).
-

2) Conceitos essenciais (o que você precisa entender antes de executar)

2.1 O que é um gateway reverso (reverse proxy)

Um **reverse proxy** é um servidor que recebe requisições da Internet e as encaminha para serviços internos.

No nosso projeto, o Nginx: - escuta na **porta 80** (e futuramente 443); - recebe requisições como `http://<IP_VPS>/users/health`; - encaminha para o container correto na rede interna, por exemplo `http://users-service:3011/health`.

Por que isso é importante? - reduz a superfície de ataque (não expõe cada microserviço diretamente); - centraliza CORS, headers de segurança, rate limit e TLS; - simplifica o deploy e a operação.

2.2 O que é Nginx

Nginx é um servidor web e reverse proxy muito usado em produção.

No nosso caso, ele faz: - roteamento por path (`/auth/`, `/users/` etc.); - rate limiting básico; - headers de segurança; - porta única pública.

2.3 O que é Certbot (e por que ele aparece neste contexto)

Certbot é uma ferramenta para emitir e renovar certificados TLS (HTTPS) via Let's Encrypt.

Neste caso, o deploy foi validado em **HTTP** (porta 80). Em evolução de projeto real, você vai: - manter o Nginx como borda; - usar Certbot para emitir certificados; - passar a servir **HTTPS na porta 443**.

Para alunos: neste momento, foque em fazer o gateway funcionar em HTTP primeiro. HTTPS vem depois.

2.4 Firewall em camadas

Em cloud, o bloqueio pode acontecer em **mais de uma camada**:

- 1) **Firewall da VM** (Linux — iptables/ufw)
- 2) **Firewall da nuvem** (Oracle Cloud — Security List / Network Security Group)

Regra prática: - Se funciona dentro da VPS (via SSH) mas não funciona externamente, **quase sempre é firewall**.

3) Problema identificado

Após o deploy dos microsserviços na VPS, o gateway Nginx estava funcionando **internamente**, mas não respondia externamente.

Sintomas observados

- Dentro da VPS:
 - `curl http://127.0.0.1/` → **200 OK** ✓
- Fora da VPS (sua máquina):
 - `curl http://<IP_VPS>/` → **timeout** ✗
- Containers rodando normalmente
- Porta 80 “aparentemente” escutando no host

4) Causa raiz: bloqueio em duas camadas

4.1 Iptables na VPS (Linux)

O firewall tinha regras permitindo apenas algumas portas (ex.: 22, 3010, 3011, 3012) e depois uma regra `REJECT all`.

Exemplo (simplificado):

```
Chain INPUT (policy ACCEPT)
1  ACCEPT    state RELATED,ESTABLISHED
2  ACCEPT    icmp
3  ACCEPT    all (loopback)
4  ACCEPT    tcp dpt:22 (SSH)
5  ACCEPT    tcp dpt:3012
6  ACCEPT    tcp dpt:3011
```

```
7 ACCEPT tcp dpt:3010
8 REJECT all ← bloqueia a porta 80
```

Aprendizado: iptables avalia regras **na ordem**. Um `REJECT all` no meio bloqueia tudo que não foi liberado antes.

4.2 Oracle Cloud Security List

A VCN não tinha regra de ingresso liberando portas **80/443** para a Internet.

Aprendizado: mesmo que a VM libere a porta, a nuvem ainda pode bloquear.

5) Solução implementada (passo a passo)

Passo 1 — Liberar 80/443 no iptables (na VPS)

Execute na VPS via SSH:

```
# Liberar HTTP (porta 80) ANTES da regra REJECT
sudo iptables -I INPUT 5 -p tcp --dport 80 -j ACCEPT

# Liberar HTTPS futuro (porta 443)
sudo iptables -I INPUT 6 -p tcp --dport 443 -j ACCEPT

# Verificar ordem e regras
sudo iptables -L INPUT -n --line-numbers

# Salvar regras (persistência após reboot)
sudo sh -c 'iptables-save > /etc/iptables.rules'
```

Observação didática: dependendo da imagem Linux, você pode precisar de um mecanismo de persistência (ex.: `iptables-persistent`). O importante é **não perder as regras após reboot**.

Passo 2 — Liberar 80/443 na Oracle Cloud Security List

1. Acessar o **Oracle Cloud Console**
2. Ir em **Networking → Virtual Cloud Networks**
3. Abrir a VCN do projeto (ex.: `vcn-aurora`)
4. Ir em **Security Lists → Default Security List**
5. Em **Ingress Rules**, clicar em **Add Ingress Rules**
6. Configurar:

Campo	Valor
Stateless	X (desmarcado)
Source Type	CIDR

Campo	Valor
Source CIDR	0.0.0.0/0
IP Protocol	TCP
Destination Port Range	80-443
Description	HTTP/HTTPS Access

1. Confirmar em **Add Ingress Rules**

Nota: pode levar alguns minutos para propagar.

6) Verificação do funcionamento

6.1 Testes mínimos

```
# Dentro da VPS
curl -s http://127.0.0.1/
# Esperado: "Aurora gateway ativo"

# Fora da VPS
curl -s http://<IP_VPS>/
# Esperado: "Aurora gateway ativo"
```

6.2 Health checks via gateway

```
curl http://<IP_VPS>/auth/health
curl http://<IP_VPS>/users/health
curl http://<IP_VPS>/events/health
curl http://<IP_VPS>/registrations/health
# Esperado: {"status":"ok"}
```

7) Workaround útil: SSH Tunnel

Se a sua rede (empresa/universidade) bloquear acessos externos, use um túnel SSH.

```
# mapeia porta local 8080 para porta 80 da VPS
ssh -f -N -L 8080:localhost:80 ubuntu@<IP_VPS>

curl http://localhost:8080/

# encerrar
pkill -f "ssh -f -N -L"
```

Por que funciona? - geralmente a porta 22 (SSH) está liberada.

8) Configuração do Nginx (o mínimo que você precisa copiar)

Arquivo: `nginx/prod.conf`

```
# Rate limiting
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=5r/s;

server {
    listen 80;
    server_name _;

    # Cabeçalhos de segurança
    add_header X-Content-Type-Options nosniff;
    add_header X-Frame-Options SAMEORIGIN;
    add_header X-XSS-Protection "1; mode=block";

    # Rota raiz
    location = / {
        return 200 'Aurora gateway ativo';
        add_header Content-Type text/plain;
    }

    # Rotas dos microsserviços
    location /auth/ {
        limit_req zone=api_limit burst=15 nodelay;
        proxy_pass http://auth-service:3010/;
    }

    location /users/ {
        limit_req zone=api_limit burst=15 nodelay;
        proxy_pass http://users-service:3011/;
    }

    location /events/ {
        limit_req zone=api_limit burst=15 nodelay;
        proxy_pass http://events-service:3012/;
    }

    location /registrations/ {
        limit_req zone=api_limit burst=15 nodelay;
        proxy_pass http://registrations-service:3013;
    }
}
```

Leituras importantes para alunos: - `proxy_pass` usa o **nome do serviço** + porta interna (DNS do Docker). - `limit_req` aplica limitação básica por IP. - A rota `/` ajuda a validar rapidamente se o gateway está “vivo”.

9) Arquitetura final (modelo mental)

```
Internet
  ↓
Oracle Cloud Security List (libera 80/443)
  ↓
iptables na VPS (ACCEPT 80/443)
  ↓
Nginx Gateway (porta 80)
  ↓
rede interna (docker network)
  ↓
auth/users/events/registrations + postgres
```

10) Checklist para você replicar no seu projeto

A seguir está um checklist **executável**, com comandos e evidências mínimas para você validar cada etapa.

Observação importante sobre placeholders: evite usar `<...>` (ex.: `<IP_VPS>`), pois alguns renderizadores interpretam isso como HTML e o texto pode “sumir”. Use `IP_VPS`, `USUARIO`, `CONTAINER_GATEWAY`.

10.1 Pré-requisitos

Infra e acesso - ☐ VPS criada e acessível por SSH (porta 22 aberta na Oracle Cloud). - Evidência: `ssh ubuntu@IP_VPS` conecta sem erro. - ☐ Docker e Docker Compose instalados na VPS. - Evidência:

```
docker --version
docker compose version
```

- ☐ Repositório clonado na VPS (ou arquivos copiados) e você está na pasta correta do projeto. - Evidência: `ls` mostra `docker-compose.deploy.yml`, `nginx/`, etc.

Configuração do projeto - ☐ `.env.prod` criado e preenchido na VPS (**não commitar**). - Evidência: `head -n 5 .env.prod` (sem expor segredos em prints públicos). - ☐ `docker-compose.deploy.yml` referencia o gateway Nginx e os serviços na mesma rede interna. - Evidência: `docker compose -f docker-compose.deploy.yml config` (valida o YAML e expande vars).

Registry (se usar GHCR) - ☐ Token/personal access token disponível na VPS (recomendado via variável `GH_PAT` no shell ou arquivo seguro). - Evidência: comando `docker login` funciona (ver seção 10.3).

10.2 Firewall (duas camadas)

A) VM (iptables)

- ☐ Porta 80 liberada **antes** de qualquer regra de bloqueio (ex.: `REJECT all`).
- ☐ Porta 443 liberada (preparação para HTTPS).
- ☐ Regras persistem após reboot (mecanismo de persistência configurado ou documentado).

Comandos de verificação (na VPS):

```
# Ver regras e ordem
sudo iptables -L INPUT -n --line-numbers

# Ver processos/portas escutando
sudo ss -lntp | grep -E ':(80|443)\b' || true
```

Se precisar adicionar as regras (na VPS):

```
sudo iptables -I INPUT 5 -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -p tcp --dport 443 -j ACCEPT
sudo iptables -L INPUT -n --line-numbers

# Persistência (exemplo simples)
sudo sh -c 'iptables-save > /etc/iptables.rules'
```

Observação: em algumas distros é recomendável instalar/usar `iptables-persistent` (ou mecanismo equivalente) para garantir persistência de forma “oficial”. O requisito pedagógico aqui é: **documentar e garantir persistência**.

B) Oracle Cloud (Security List / NSG)

- ☐ Regra de ingresso liberando **TCP 80 e 443** (geralmente para `0.0.0.0/0`).
- ☐ Regra de SSH (TCP 22) liberada apenas para sua origem (boa prática) ou para `0.0.0.0/0` se não houver alternativa.

Evidências mínimas: - print/config da regra com porta `80-443` habilitada. - após a regra, teste externo (seção 10.4) retorna resposta.

10.3 Deploy (gateway + serviços)

(Opcional) Login no GHCR

Na VPS:

```
echo "$GH_PAT" | docker login ghcr.io -u USUARIO --password-stdin
```

Pull das imagens e subida do stack

Na VPS (ajuste o arquivo de compose conforme seu projeto):

```
docker compose --env-file .env.prod -f docker-compose.deploy.yml pull

docker compose --env-file .env.prod -f docker-compose.deploy.yml up -d

docker compose --env-file .env.prod -f docker-compose.deploy.yml ps
```

Evidências mínimas: - `ps` mostra gateway Nginx e serviços como `Up`. - se algum container estiver reiniciando, coletar logs.

Coleta rápida de logs (na VPS):

```
# gateway (ajuste o nome do serviço conforme o compose)
docker compose --env-file .env.prod -f docker-compose.deploy.yml logs --
tail=200 gateway

# exemplo para um microserviço
docker compose --env-file .env.prod -f docker-compose.deploy.yml logs --
tail=200 users-service
```

10.4 Verificação (interno e externo)

A) Teste interno (dentro da VPS)

```
curl -s -i http://127.0.0.1/
# Esperado: 200 e texto "Aurora gateway ativo"
```

Se falhar internamente: - verifique se o gateway está escutando: `sudo ss -lntp | grep ':80'` - verifique logs do container do gateway.

B) Teste externo (da sua máquina)

```
curl -s -i http://IP_VPS/  
# Esperado: 200 e texto "Aurora gateway ativo"
```

Se falhar externamente e funcionar internamente: - trate como problema de firewall em camadas (seção 10.2).

C) Health checks via gateway

```
curl -s -i http://IP_VPS/auth/health  
curl -s -i http://IP_VPS/users/health  
curl -s -i http://IP_VPS/events/health  
curl -s -i http://IP_VPS/registrations/health
```

Evidência: - HTTP 200 em cada endpoint e payload esperado (ex.: `{"status": "ok"}`).

10.5 Troubleshooting rápido (quando algo dá errado)

- **Funciona** `127.0.0.1` **na VPS, mas timeout externo** → firewall (Oracle + iptables).
- **Não funciona nem** `127.0.0.1` **na VPS** → gateway/container/config.
- ver `docker compose ... ps`
- ver logs do gateway
- confirmar porta 80 mapeada no host (no compose) e serviço escutando.
- **Gateway responde** `/`, **mas** `/users/health` **falha** → problema de roteamento/serviço interno.
- testar do gateway para o serviço por DNS interno:

```
docker exec -it CONTAINER_GATEWAY sh  
# dentro do container  
wget -qO- http://users-service:3011/health || true
```

11) Lições aprendidas

1. **Firewall em camadas:** VM e nuvem podem bloquear ao mesmo tempo.
2. **Ordem no iptables:** `REJECT all` no meio bloqueia tudo.
3. **Debug sistemático:**
4. funciona interno e falha externo → firewall
5. falha interno → aplicação/container/config
6. **Automação ajuda:** scripts/workflows reduzem erro humano.
7. **SSH tunnel é plano B** em redes restritivas.

12) Próximo passo recomendado (HTTPS com Certbot)

Quando o gateway em HTTP estiver estável: - configurar um domínio (DNS apontando para a VPS); - usar **Certbot** para emitir certificados; - atualizar Nginx para `listen 443 ssl;` e redirecionar 80 → 443; - validar renovação automática.

Importante: HTTPS depende de domínio (em geral) para emissão fácil do certificado.

Status

Concluído em 16/12/2025: -  Gateway Nginx operacional na VPS -  Endpoints acessíveis via gateway -  Firewall (iptables + Oracle Cloud) configurado -  Checklist e passos replicáveis para alunos